End-to-End CI/CD Pipeline

GitHub -> Docker -> Jenkins -> AWS EC2 (Amazon Linux 2023)

Deployment Guide

August 2026

Contents

Before You Start: What We Are Building 1

Step 1 — Create the GitHub Repository 2

Step 2 — Create the Website Files (HTML + CSS) 2

Step 3 — Write the Dockerfile 3

Step 4 — Build and Test the Docker Image Locally 4

Step 5 — Push the Image to Docker Hub 4

Step 6 — Launch the AWS EC2 Instance (Amazon Linux 2023) 5

Step 7 — Connect and Install Docker on the EC2 Instance 5

Step 8 — Manually Run the Container on EC2 (Sanity Check) 6

Step 9 — Install Jenkins (Skip This If Jenkins Already Exists) 6

Step 10 — Add Credentials in Jenkins 7

Step 11 — Create the Jenkins Pipeline Job 7

Step 12 — Automatic Trigger on Every GitHub Push (Optional but Recommended) 9

Step 13 — Final Verification 9

Troubleshooting Quick Reference 9

One-Page Command Summary 10

Before You Start: What We Are Building

We are building a small **static website (HTML + CSS)** and automating its full delivery path:

```
GitHub repo  --->  Docker image  --->  Docker Hub  --->  Jenkins Pipeline  --->  AWS EC2 (Amazon Linux 2023)
(source code)    (built by you      (image        (builds + pushes +    (pulls image, runs
                 or Jenkins)        storage)      deploys automatically)  container on port 84)
```

Assumption about the port: you said the site "should work in eighty four." This guide maps your container's internal port **80** (where the web server listens) to **host port 84** on the EC2 instance. That means once deployed, you will open the site at:

`http://<EC2-PUBLIC-IP>:84`

If you actually meant a different port (8080, 8084, etc.), just replace every 84 in this guide with the port number you want — the steps do not otherwise change.

Tools/accounts you need before starting:

- A GitHub account (free) — https://github.com
- A Docker Hub account (free) — https://hub.docker.com
- An AWS account with billing enabled — https://aws.amazon.com
- Docker installed on your own laptop (Windows/Mac/Linux) — https://docs.docker.com/get-docker
- Git installed on your own laptop — https://git-scm.com/downloads
- A Jenkins server (this guide includes how to install one if you don't have one yet)

Step 1 — Create the GitHub Repository

1. Go to **https://github.com** and log in.
2. Click the **+** icon (top right) → **New repository**.
3. Fill in:
   - **Repository name**: `my-static-site` (you can rename this, but use this name consistently through the rest of the guide, since Jenkins and Docker will reference it).
   - **Visibility**: Public or Private, your choice.
   - Check **Add a README file**.
4. Click **Create repository**.
5. On your laptop, open a terminal and clone it:

```
git clone https://github.com/<your-username>/my-static-site.git
cd my-static-site
```

Replace `<your-username>` with your actual GitHub username everywhere in this guide.

Step 2 — Create the Website Files (HTML + CSS)

Inside the `my-static-site` folder, create two files.

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My CI/CD Website</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>Hello from my Dockerized Website!</h1>
```

```
    <p>Delivered via GitHub &rarr; Docker &rarr; Jenkins &rarr; AWS EC2</p>
  </div>
</body>
</html>
```

style.css

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #1e1e2f;
  color: #ffffff;
  display: flex;
  align-items: center;
  justify-content: center;
  height: 100vh;
}
.container {
  text-align: center;
}
h1 {
  color: #4fd1c5;
}
```

Push these two files to GitHub:

```
git add index.html style.css
git commit -m "Add static website files"
git push origin main
```

Line-by-line meaning: `git add` stages the new files, `git commit` saves a snapshot with a message, `git push origin main` uploads that snapshot to GitHub's `main` branch.

Step 3 — Write the Dockerfile

In the same folder, create a file literally named `Dockerfile` (no extension):

```
# Use the latest stable, lightweight NGINX web server image
FROM nginx:stable-alpine

# Remove the default NGINX welcome page
RUN rm -rf /usr/share/nginx/html/*

# Copy our website files into NGINX's web root
COPY index.html /usr/share/nginx/html/
COPY style.css /usr/share/nginx/html/

# Document that the container listens on port 80 internally
EXPOSE 80

# Default command that starts NGINX in the foreground
CMD ["nginx", "-g", "daemon off;"]
```

What each line does: - `FROM nginx:stable-alpine` — starts from the official, current, lightweight NGINX image (Alpine Linux base, small and secure). - `RUN rm -rf ...` — deletes NGINX's default sample page so only your site shows. - `COPY` — copies your two files into the folder NGINX serves by default. - `EXPOSE 80` — documents that the app listens on port 80 inside the container (this does not open ports by itself; that happens later with `-p` or the security group). - `CMD` — the command that runs when the container starts; `daemon off` keeps NGINX running in the foreground so Docker can track the process.

Push this file too:

```
git add Dockerfile
git commit -m "Add Dockerfile"
git push origin main
```

Step 4 — Build and Test the Docker Image Locally

Still in the `my-static-site` folder on your laptop:

```
docker build -t my-static-site:latest .
```

- `docker build` — tells Docker to build an image.
- `-t my-static-site:latest` — tags (names) the image `my-static-site` with version `latest`.
- `.` — tells Docker to use the current folder (where the Dockerfile is) as the build context.

Run it locally to confirm it works before touching Docker Hub or AWS:

```
docker run -d -p 8888:80 --name test-site my-static-site:latest
```

- `-d` — run in the background (detached).
- `-p 8888:80` — maps your laptop's port 8888 to the container's port 80.
- `--name test-site` — names the running container for easy reference.

Open `http://localhost:8888` in a browser — you should see your page. Then clean up:

```
docker stop test-site
docker rm test-site
```

Step 5 — Push the Image to Docker Hub

1. Create a repository on Docker Hub:
   - Go to **https://hub.docker.com** → **Create Repository**.
   - Name it `my-static-site`, set it Public, click **Create**.
2. Log in from your terminal:

```
docker login
```

Enter your Docker Hub username and password (or access token) when prompted.

3. Tag and push the image:

```
docker tag my-static-site:latest <dockerhub-username>/my-static-site:latest
docker push <dockerhub-username>/my-static-site:latest
```

- `docker tag` — relabels your local image with your Docker Hub username so Docker knows where to send it.
- `docker push` — uploads the image to Docker Hub, making it pullable from anywhere (including your future EC2 server and Jenkins).

This manual push confirms your pipeline's core artifact works end to end before we automate it with Jenkins in Step 10.

Step 6 — Launch the AWS EC2 Instance (Amazon Linux 2023)

1. Log into the **AWS Console** → search **EC2** → **Launch instance**.
2. **Name**: `my-static-site-server`.
3. **Application and OS Images**: choose **Amazon Linux** → select **Amazon Linux 2023 AMI** (this is the current, supported Amazon Linux release; AWS keeps the AMI itself patched automatically each time a new AMI version is published).
4. **Instance type**: `t2.micro` (free-tier eligible) or `t3.micro`.
5. **Key pair**: click **Create new key pair**, name it `my-site-key`, type RSA, format `.pem`, download and save it somewhere safe (e.g., `~/keys/my-site-key.pem`). You need this file to SSH into the server and for Jenkins to deploy to it later.
6. **Network settings** → click **Edit**:
   - Allow **SSH (port 22)** from "My IP" (safer than "Anywhere").
   - Click **Add security group rule**:
     - Type: Custom TCP
     - Port range: 84
     - Source: Anywhere (0.0.0.0/0) — so the website is publicly reachable on port 84.
7. Leave storage at default (8 GiB is enough for this).
8. Click **Launch instance**.
9. Once the instance state shows "Running," note its **Public IPv4 address** — you'll need it constantly from here on.

Step 7 — Connect and Install Docker on the EC2 Instance

From your laptop terminal:

```
chmod 400 ~/keys/my-site-key.pem
ssh -i ~/keys/my-site-key.pem ec2-user@<EC2-PUBLIC-IP>
```

- `chmod 400` — restricts the key file's permissions (SSH refuses to use overly open key files).
- `ssh -i <key> ec2-user@<ip>` — connects to the server; `ec2-user` is the default login user on Amazon Linux AMIs.

Once connected, install Docker (Amazon Linux 2023 uses `dnf`, not `yum`/`amazon-linux-extras` like older Amazon Linux 2 did):

```
sudo dnf update -y
sudo dnf install docker -y
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker ec2-user
```

- `dnf update -y` — refreshes and updates all installed packages to their current versions.
- `dnf install docker -y` — installs Docker Engine from Amazon Linux's own repositories.
- `systemctl enable docker` — makes Docker start automatically on every reboot.
- `systemctl start docker` — starts the Docker service right now.
- `usermod -aG docker ec2-user` — adds your login user to the `docker` group so you don't need `sudo` for every docker command.

Log out and back in (or run `newgrp docker`) for the group change to apply:

```
exit
ssh -i ~/keys/my-site-key.pem ec2-user@<EC2-PUBLIC-IP>
docker --version
```

Step 8 — Manually Run the Container on EC2 (Sanity Check)

While still connected to the EC2 instance:

```
docker pull <dockerhub-username>/my-static-site:latest
docker run -d -p 84:80 --restart unless-stopped --name my-static-site <dockerhub-username>
```

- `docker pull` — downloads your image from Docker Hub onto the EC2 server.
- `-p 84:80` — maps EC2's port 84 (external) to the container's port 80 (internal), matching the security group rule from Step 6.
- `--restart unless-stopped` — automatically restarts the container if it crashes or the server reboots.
- `--name my-static-site` — a friendly name so you can reference it in future commands (`docker stop my-static-site`, `docker logs my-static-site`, etc.).

Now open a browser and go to:

`http://<EC2-PUBLIC-IP>:84`

You should see your website. This confirms the full manual chain works. Steps 9–12 now make Jenkins do Steps 5 and 8 automatically whenever you push new code to GitHub.

Step 9 — Install Jenkins (Skip This If Jenkins Already Exists)

You can run Jenkins on the same EC2 instance or, better for a real class/production setup, a **separate** EC2 instance (also Amazon Linux 2023) so Jenkins doesn't compete with your website's resources. The commands below assume a separate instance reachable via SSH the same way as Step 7.

```
sudo dnf update -y
sudo dnf install java-17-amazon-corretto -y
sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.re
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
sudo dnf install jenkins -y
sudo systemctl enable jenkins
sudo systemctl start jenkins
```

- Jenkins (current LTS) requires **Java 17**; `java-17-amazon-corretto` is Amazon's supported OpenJDK build.
- The `wget`/`rpm --import` lines add Jenkins's official package repository and signing key so `dnf` can find and trust the `jenkins` package.
- `systemctl enable/start jenkins` makes it run now and on every reboot.

Also install Docker on this Jenkins server (Jenkins needs Docker locally to build images):

```
sudo dnf install docker -y
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG jenkins docker
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
```

Open the security group for this instance and add an inbound rule for **TCP port 8080** (Jenkins's default web UI port) from "My IP."

Get the initial admin password and finish setup in the browser:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Go to `http://<JENKINS-SERVER-IP>:8080`, paste that password, choose **Install suggested plugins**, then create your admin user when prompted.

Step 10 — Add Credentials in Jenkins

In the Jenkins dashboard:

1. **Manage Jenkins → Credentials → System → Global credentials → Add Credentials**
2. **Docker Hub credential:**
   - Kind: **Username with password**
   - Username: your Docker Hub username
   - Password: your Docker Hub password or access token
   - ID: `dockerhub-creds` (used exactly by name in the Jenkinsfile below)
3. **EC2 SSH credential** (so Jenkins can deploy to your website server):
   - Kind: **SSH Username with private key**
   - Username: `ec2-user`
   - Private key: **Enter directly**, paste the full contents of your `my-site-key.pem` file
   - ID: `ec2-ssh-key`

Also install two plugins if not already present (**Manage Jenkins → Plugins → Available**): - **Docker Pipeline** - **SSH Agent**

Step 11 — Create the Jenkins Pipeline Job

1. Add a `Jenkinsfile` to your GitHub repo (same folder as the Dockerfile):

```
pipeline {
    agent any

    environment {
```

```
        DOCKERHUB_CREDENTIALS = credentials('dockerhub-creds')
        IMAGE_NAME = "<dockerhub-username>/my-static-site"
        EC2_HOST = "<EC2-PUBLIC-IP>"
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build Docker Image') {
            steps {
                sh "docker build -t ${IMAGE_NAME}:${BUILD_NUMBER} -t ${IMAGE_NAME}:latest
            }
        }

        stage('Push to Docker Hub') {
            steps {
                sh "echo $DOCKERHUB_CREDENTIALS_PSW | docker login -u $DOCKERHUB_CREDENTIA
                sh "docker push ${IMAGE_NAME}:${BUILD_NUMBER}"
                sh "docker push ${IMAGE_NAME}:latest"
            }
        }

        stage('Deploy to EC2') {
            steps {
                sshagent(credentials: ['ec2-ssh-key']) {
                    sh """
                        ssh -o StrictHostKeyChecking=no ec2-user@${EC2_HOST} '
                          docker pull ${IMAGE_NAME}:latest &&
                          docker stop my-static-site || true &&
                          docker rm my-static-site || true &&
                          docker run -d -p 84:80 --restart unless-stopped --name my-static
                        '
                    """
                }
            }
        }
    }

    post {
        always {
            sh 'docker logout'
        }
    }
}
```

What each stage does: - **Checkout** — pulls the exact commit that triggered the build from GitHub. - **Build Docker Image** — rebuilds the image using the same Dockerfile from Step 3, tagging it with both the Jenkins build number (for history/rollback) and `latest`. - **Push to Docker Hub** — logs in using the stored credential and uploads both tags. - **Deploy to EC2** — uses the stored SSH key to log into your website server and replace the running container

with the freshly pushed image, exactly like the manual commands in Step 8. - **post/always** — logs Docker out of Docker Hub at the end regardless of success or failure, so credentials aren't left cached on disk.

Commit and push this file:

```
git add Jenkinsfile
git commit -m "Add Jenkins pipeline"
git push origin main
```

2. In Jenkins: **New Item** → name it `my-static-site-pipeline` → choose **Pipeline** → **OK**.
3. Under **Pipeline** section:
   - **Definition**: `Pipeline script from SCM`
   - **SCM**: `Git`
   - **Repository URL**: `https://github.com/<your-username>/my-static-site.git`
   - **Branch**: `*/main`
   - **Script Path**: `Jenkinsfile`
4. Click **Save**, then **Build Now** to run it the first time manually.

Step 12 — Automatic Trigger on Every GitHub Push (Optional but Recommended)

1. In your Jenkins job configuration, under **Build Triggers**, check **GitHub hook trigger for GITScm polling**.
2. In your GitHub repo: **Settings → Webhooks → Add webhook**:
   - **Payload URL**: `http://<JENKINS-SERVER-IP>:8080/github-webhook/`
   - **Content type**: `application/json`
   - **Events**: Just the push event
   - Click **Add webhook**.

From now on, every `git push` to `main` automatically triggers Jenkins to build, push, and redeploy your site — no manual steps needed.

Step 13 — Final Verification

`http://<EC2-PUBLIC-IP>:84`

Push a small change (e.g., edit the <h1> text in `index.html`), commit, and push:

```
git add index.html
git commit -m "Update homepage text"
git push origin main
```

Within moments, Jenkins should show a new build running, and refreshing the site in your browser should show the updated text — confirming the entire pipeline works end to end.

Troubleshooting Quick Reference

Symptom	Likely Cause	Fix
Site unreachable on port 84	Security group missing rule	Add inbound custom TCP 84, source 0.0.0.0/0
`docker: permission denied` on EC2	User not in `docker` group yet	Re-login or run `newgrp docker`
Jenkins build fails at `docker login`	Wrong/expired Docker Hub credential	Update the `dockerhub-creds` credential in Jenkins
Jenkins `Deploy to EC2` stage fails to SSH	Wrong key, or EC2 security group blocks Jenkins's IP on port 22	Confirm `ec2-ssh-key` credential content and open port 22 to the Jenkins server's IP
Webhook doesn't trigger builds	Jenkins URL not publicly reachable, or wrong payload URL	Ensure Jenkins security group allows inbound 8080 from GitHub, double check webhook URL ends in `/github-webhook/`
Old page still showing after deploy	Browser cache	Hard refresh (Ctrl+Shift+R)

One-Page Command Summary

```
# Local machine
git clone https://github.com/<user>/my-static-site.git
docker build -t my-static-site:latest .
docker run -d -p 8888:80 --name test-site my-static-site:latest
docker login
docker tag my-static-site:latest <dockerhub-user>/my-static-site:latest
docker push <dockerhub-user>/my-static-site:latest

# EC2 website server (Amazon Linux 2023)
sudo dnf update -y && sudo dnf install docker -y
sudo systemctl enable docker && sudo systemctl start docker
sudo usermod -aG docker ec2-user
docker pull <dockerhub-user>/my-static-site:latest
docker run -d -p 84:80 --restart unless-stopped --name my-static-site <dockerhub-user>/my-

# Jenkins server (Amazon Linux 2023)
sudo dnf install java-17-amazon-corretto docker -y
sudo dnf install jenkins -y
sudo systemctl enable jenkins && sudo systemctl start jenkins
```